UNIT- IV

Advanced Behavior Modeling:

Events and Signals

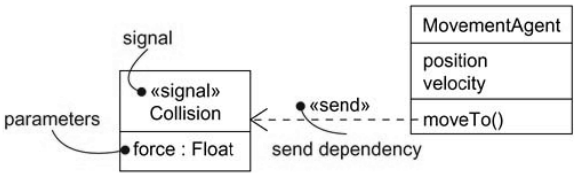
- **An event** is the specification of a significant occurrence that has a location in **time and space**.
- **A signal** is a kind of event that represents the specification of things communicated between instances.

Kinds of Events

- **Events may be external or internal.**
- **External events** are those that **pass between the system and its actors**.
- **Internal events** are those that pass **among the objects** that live inside the system.
- In the UML, you can model **four kinds of events**: **signals, calls(call event)**, the **passing of time**, and a **change in state.**

Signals

- A **signal represents a named object that is dispatched (thrown) asynchronously by one object and then received (caught) by another.**
- **Signals may also be involved in generalization relationships, permitting you to model hierarchies of events**, **some o**f which **are general** and **some of which are specific Also as for classes, signals may have attributes and operations.**
- A **signal may be sent as the action of a state transition in a state machine** or **the sending of a message in an interaction.**
- **The execution of an operation can also send signals.**
- **We model signals (and exceptions**) as stereotyped classes. We can use a dependency, stereotyped as <<**send>>**, to indicate that an operation sends a particular signal.

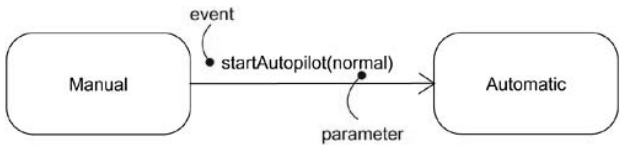


Signals

Call Events

- **Just as a signal event represents the occurrence of a signal,**
- a **call** event represents **the dispatch of an operation**.
- In both cases, the event may trigger a state transition in a state machine
- Whereas a **signal is an asynchronous event**, a **call event is synchronous**
- **This means that when an object invokes an operation on another object that has a state machine, control passes from the sender to the receiver.**

- the transition is triggered by the event, the operation is completed, the receiver transitions to a new state, and control returns to the sender.
- In both cases, you show the event, **along with its parameters**, as the trigger for a state transition.



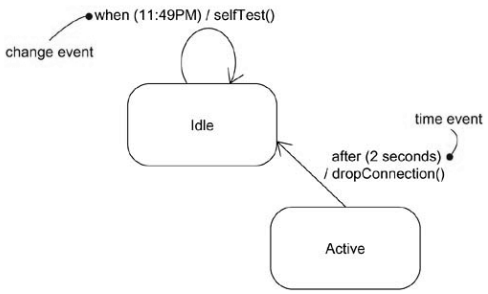
Call Events

Time and Change Events

- **A time event** is an **event that represents the passage of time**
- in the UML you model a time event by using the **keyword after followed by some expression that evaluates to a period of time.**
- Unless you **specify it explicitly**, the **starting time of such an expression is the time since entering the current state.**

Change Event:

- **A change event** is an event that **represents a change in state or the satisfaction of some condition**
- In the UML you model a change event by using **the keyword when followed by some Boolean expression**
- You can use such expressions to mark an absolute time (such as when time = 11:59) or for the continuous test of an expression

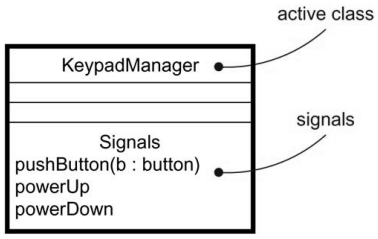


Time and Change Events

Sending and Receiving Events

- Signal events and call events involve at least two objects:
    - The object that sends the signal or invokes the operation
    - The object to which the event is directed.
- Any instance of any class can send a signal to or invoke an operation of a receiving object.
- When an object **sends a signal, the sender dispatches the signal and then continues along its flow of control, not waiting for any return from the receiver.**

- Any instance of any class can receive a call event or a signal.
- If this is a synchronous call event, then the sender and the receiver for the duration of the operation.



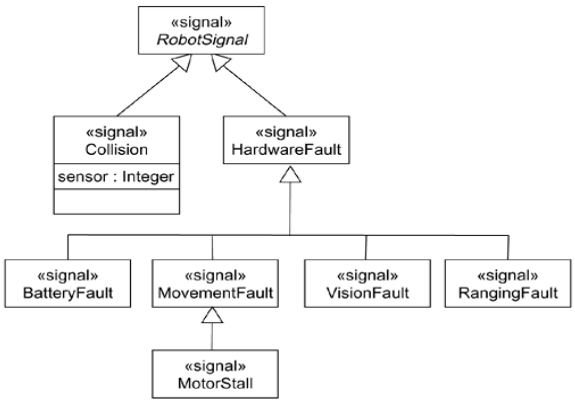
Signals and Active Classes

Common Modeling Techniques:

1. Modeling a Family of Signals

- In most event-driven systems, signal events are hierarchical.
- External and internal signals need not be disjoint, however. Even within these two broad classifications, you might find specializations

- To model a family of signals
  - Consider all the different kinds of signals to which a given set of active objects may respond.
  - Look for the common kinds of signals and place them in a generalization/specialization hierarchy using inheritance. Elevate more general ones and lower more specialized ones.
  - Look for the opportunity for polymorphism in the state machines of these active objects. Where you find polymorphism, adjust the hierarchy as necessary by introducing intermediate abstract signals.



2. Modeling Exceptions

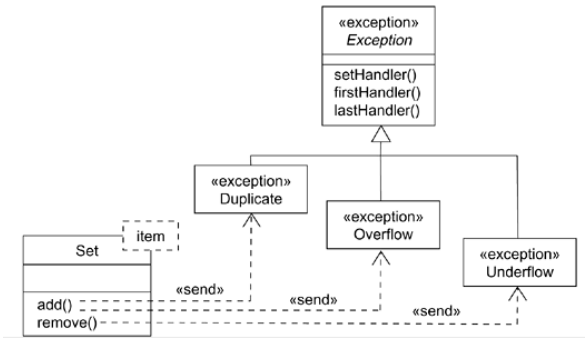
- An important part of visualizing, specifying, and documenting the behavior of a class or an interface is specifying the exceptions that its operations can raise.
- In the UML, exceptions are kinds of signals, which you model as stereotyped classes. Exceptions may be attached to specification operations.
- Modeling exceptions is somewhat the inverse of modeling a general family of signals.
- We model a family of signals primarily to specify the kinds of signals an active object may receive
- We model exceptions primarily to specify the kinds of exceptions that an object may throw through its operations

- To model exceptions

    - For each class and interface, and for each operation of such elements, consider the exceptional conditions that may be raised.

    - Arrange these exceptions in a hierarchy. Elevate general ones, lower specialized ones, and introduce intermediate exceptions, as necessary.

    - For each operation, specify the exceptions that it may raise. You can do so explicitly (by showing send dependencies from an operation to its exceptions) or you can put this in the operation's specification.



Modeling Exceptions

Processes and Threads

- **A process** is a **heavyweight** flow that can **execute concurrently with other processes.**
- **A thread** is a **lightweight** flow that can **execute concurrently with other threads within the same process.**
- **Graphically, an active class is rendered as a rectangle with thick lines.**
- **Processes and threads are rendered as stereotyped active classes**

Flow of Control

sequential system:

- In a purely **sequential system, there is one flow of control.**

  This means that **one thing, and one thing only, can take place at a time.**

  When a **sequential program starts, control is rooted at the beginning of the program and operations are dispatched one after another.**

- A **sequential program will process only one event at a time**.

concurrent system:

- In a **concurrent system, there is more than one flow of control**.

  that is, **more than one thing can take place at a time.**

  In a **concurrent system, there are multiple simultaneous flows of control,** each rooted at the head of an **independent process** or a thread.

Note: You can achieve true concurrency in one of **three ways**:

- **By distributing active objects across multiple nodes**
- **By placing active objects on nodes with multiple processors**
- By a combination **of both methods.**

Classes and Events

- **Active classes are just classes,** with a very special property.
- An **active class represents an independent flow of control**, whereas **Normal class no such flow**.
- You use **active classes to model common families of processes or threads.**
- **By modeling concurrent systems** with active objects, you **give a name to each independent flow of control.**
    - **When an active object is created,** the **associated flow of control is started;**
    - **When the active object is destroyed**, the **associated flow of control is terminated.**
- **Active classes share the same properties as all other classes**.
- **Active classes may have instances**.
- Active classes may have **attributes and operations**.
- Active classes may participate in **dependency, generalization, and association relationships**.
- Active classes may use any of the UML's **extensibility mechanisms, including stereotypes, tagged values, and constraints.**

Standard Elements

- All of the UML's **extensibility mechanisms apply to active classes.** you'll use tagged values to extend active class properties.
- The **UML defines two standard stereotypes that apply to active classes**.

1. process	Specifies a **heavyweight flow** that can **execute concurrently with other processes**
2. thread	Specifies a **lightweight flow** that can **execute concurrently with other threads within the same process**

- A **process is heavyweight, Each program runs as a process in its own address space**.
- **Processes are never nested inside one another**.
- If the **node has multiple processors**, then **concurrency on that node is possible**.
- If the **node has only one processor**, there is only the illusion of **true concurrency** of the system.
- A **thread is lightweight**.
- Threads are **never nested inside one another**.

Communication

- When **objects collaborate with one another**.
- they **interact by passing messages from one to the other**.
- **In a system** with both **active and passive objects,**

  there are **4 possible combinations of interaction** that you must consider

  1. A **message may be passed from <u>one passive object to another passive object</u>.**

     (Assuming **there is only one flow of control passing through these objects <u>at a time</u>**)

  2. A **message may be passed from <u>one active object to another active object</u>.**

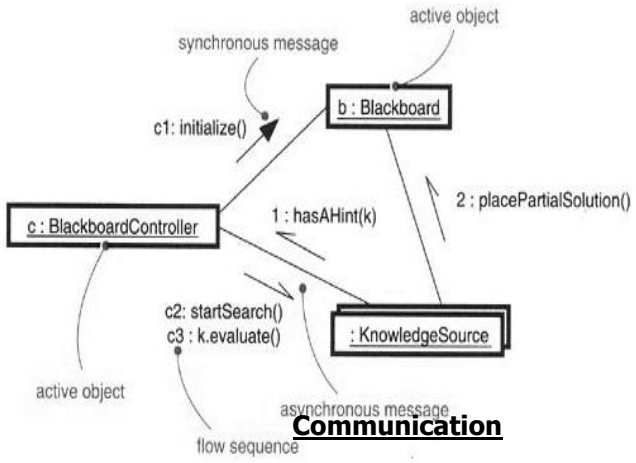
  3. When that happens, you have **interprocess communication**, and there are two possible styles of communication

     1.) **One active object might <u>synchronously</u> call an operation of another.**

     2.) **One active object might <u>asynchronously</u> send a signal or call an operation of another object**

     T

- In the UML, we render a **synchronous message as a <u>full arrow</u>** and an **asynchronous message as a <u>half arrow</u>**



<u>Communication</u>

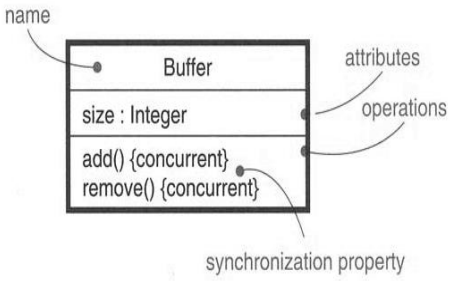
  4. **A message may be passed from an <u>active object to a passive object</u>.**

     A message may be passed from a **passive object to an active one**.

Synchronization

- The problem arises when **more than one flow of control is in one object at the same time**.
- If **you are not careful,** anything **more than one flow** with another, **corrupting the state of the object.**
- In object-oriented systems is by treating an object as a **critical region.(more than one flow of control)**
- **There are three alternatives to this approach**.
- In the UML, you can **model all three approaches.**

1. Sequential	* **Callers(sender) must coordinate outside the object so that only one flow is in the object at a time.** • Similarly **multiple flows of control,** the **semantics (meaning)and integrity of the object cannot be guaranteed.**
2. Guarded	The **semantics and integrity of the object is guaranteed (i.e --** in the presence of **multiple flows of control by sequential all calls to all of the object's guarded operations.** *(multiple flows of control with sequential)* In effect, **exactly one operation at a time can be invoked on the object**, **reducing this to sequential semantics.** (in general one object at time if more than one to follow guarded i.e multiple flows)
3. Concurrent	The semantics and integrity of the object is guaranteed in the presence of multiple flows of control by treating the operation as atomic.*(multiple flows parallel execution)*



Synchronization

Process Views

- **Active objects play an important role** in

  Visualizing,
  Specifying,
  Constructing, and
  Documenting a system's process view.

- The **process view of a system encompasses** (combine or hold with in) the **threads and processes** that form the **system's concurrency and synchronization mechanisms.**

- This view primarily addresses the

  Performance,
  Scalability,
  Throughput of the system.

  With the UML, the **static** and **dynamic aspects** of this **view are captured**.

  In the same kinds of diagrams as for **Example** design view—that is,

  Class diagrams,
  Interaction diagrams,
  Activity diagrams, and
  Statechart diagrams.

- But with a focus on the active classes that represent these threads and processes.

Common Modeling Techniques:

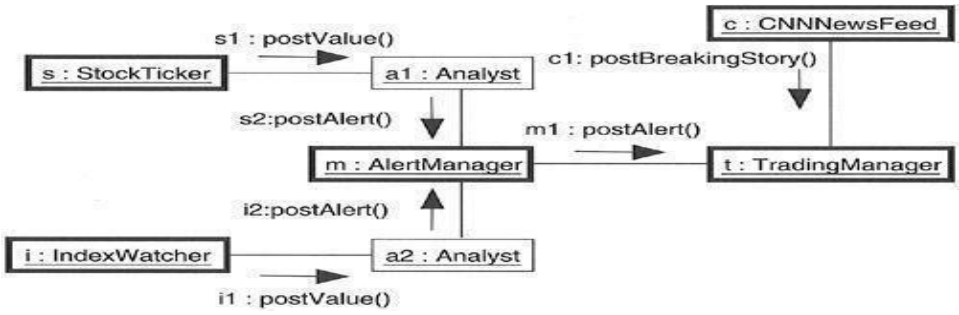
1.Modeling Multiple Flows of Control

- **Identify the actions and notations each flow as an active class**.
- Consider a **balanced distribution of responsibilities among these active classes**.
- Then **examine the active and passive classes** with which **each collaborates statically**.
- Ensure that **each active class is both tightly cohesive and loosely coupled relative to these neighboring classes** and that each has the right **set of attributes, operations, and signals**.

Example for

- **Capture these static decisions in class diagrams** and each active class.
- **Capture those decisions in interaction diagrams dynamically**.
- To communication among active objects. **Apply synchronous and asynchronous messaging**, as appropriate.

Figure Modeling Flows of Control

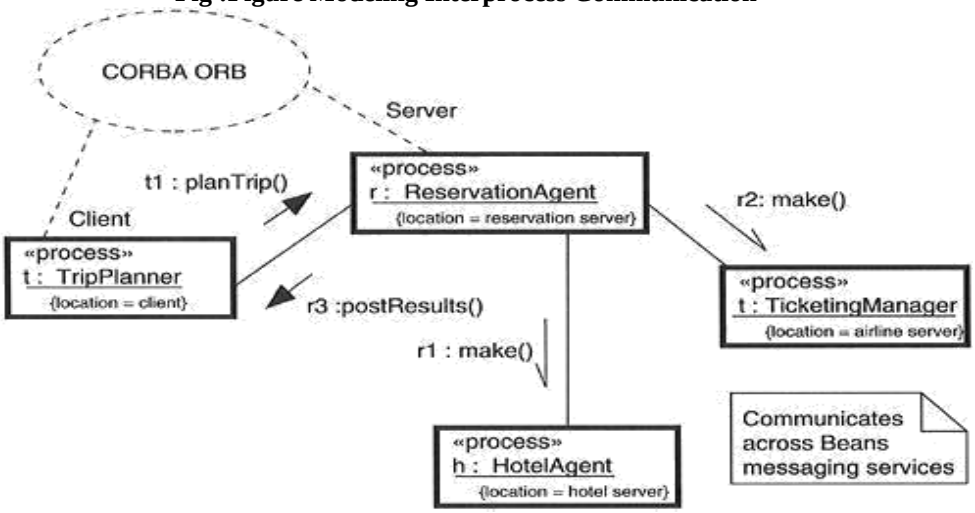


2.Modeling Interprocess Communication:

To model interprocess communication,

- Model the **multiple flows of control**.
- Consider **which represent processes** and **which represent threads**.
- Distinguish them using the **appropriate stereotype.**
- Model messaging using **asynchronous and synchronous communication**.
- Informally **specify by using notes**, or more formally by using collaborations.

Fig :Figure Modeling Interprocess Communication



Time and Space:

Terms and Concepts:

Time

Real time systems are, by their very name, time-critical systems.

Events may happen at regular or irregular times; The response to an event must happen at predictable (expected or absolute times) times relative to the event itself.

- The **passing of messages represents the dynamic aspect of any system.**
- **In UML, you can give a name to each message in an interaction to be used as a timing mark**.
- **They are mainly rendered with the name of an event,** such as a **signal or a call A timing mark** .
- That is, **startTime, stopTime, and executionTime.**

You can use these **functions to specify complex time expressions**, **either constants or variables (as long as those variables can be bound at execution time).**

Finally, as shown in Figure , you can place these time expressions in a timing constraint to specify the timing behavior of the system. As constraints, you can render them by placing them adjacent to the appropriate message, or you can explicitly attach them using dependency relationships.

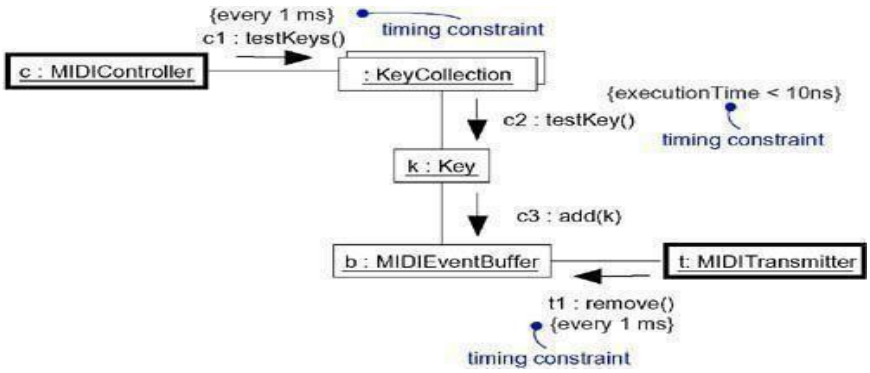
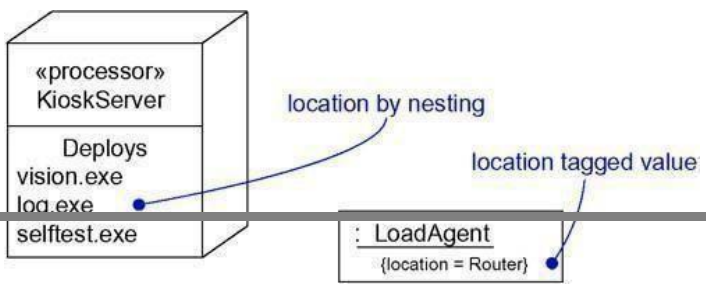


Figure Time

Location:

Distributed systems, by their nature, encompass (surrounded or closely related)) components that are physically scattered (separate or break up)among the nodes of a system.

For many systems, components are fixed in place at the time they are loaded on the system; i**n other systems, components may migrate(converted or moved) from node to node**.



Common Modeling Techniques:

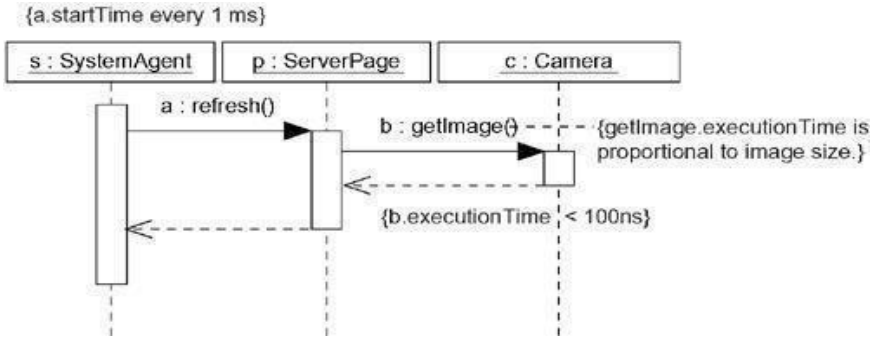
1. Modeling Timing Constraints

For each event in an interaction, consider whether it must start at some absolute time. Model that real time property as a timing constraint on the message.

For each interesting sequence of messages in an interaction, consider whether there is an associated maximum relative time for that sequence. Model that real time property as a timing constraint on the sequence.

For each time critical operation in each class, consider its time complexity. Model those semantics as timing constraints on the operation.

Figure Modeling Timing Constraint



2.Modeling the Distribution of Objects:

- For each interesting class of objects in your system, consider its locality of reference. In other words, consider all its neighbors and their locations.

  A tightly coupled locality will have neighboring objects close by; a loosely coupled one will have distant objects (and thus, there will be latency in communicating with them).

- Next consider patterns of interaction among related sets of objects. Co-locate sets of objects that have high degrees of interaction, to reduce the cost of communication. Partition sets of objects that have low degrees of interaction.

- Next consider the distribution of responsibilities across the system. Redistribute your objects to balance the load of each node.

- Consider also issues of security, volatility, and quality of service, and redistribute your objects as appropriate.

- Render this allocation in one of two ways:

  1. By nesting objects in the nodes of a deployment diagram

  2. By explicitly indicating the location of the object as a tagged value

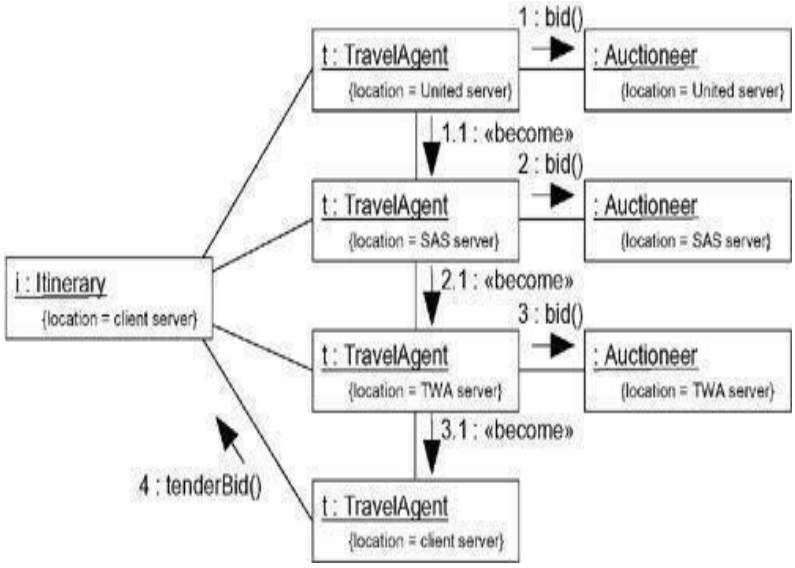


Figure Modeling the Distribution of Objects

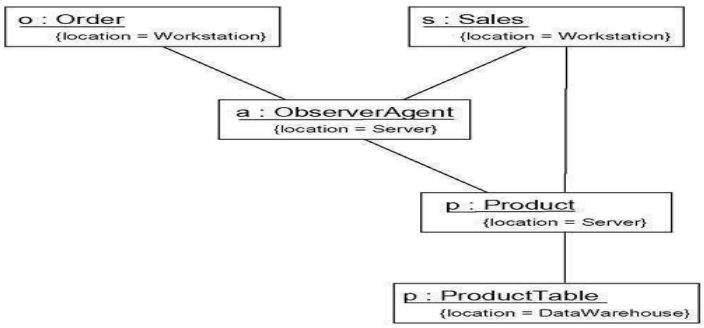
3,Modeling Objects that Migrate:

For many **Distributed systems, components and objects, once loaded on the system.**

For their **lifetime, from creation to destruction**, they never leave the node on which they were born.

There are **certain classes of distributed systems,** however, for **which things move** usually for one of **two reasons.**

- First, you'll **find objects migrating in order to move closer to actors and other objects they need to work with to do their job better.**
- Second, you'll **find objects migrating** in **response to the failure of a node** to **balance the load across multiple nodes**.



Statechart Diagrams

Terms and Concepts:

- **A *statechart diagram* shows a state machine, emphasizing the flow of control from state to state.**

- A "***state machine* is a behavior"** that

specifies the "sequences of states" an object goes through during its lifetime.

- **"A *state* is a condition or situation in the life of an object during which it satisfies some condition, performs some activity.**

- **"An *event* is the specification of a significant occurrence that has a location in time and space.**

- **Event is an occurrence of a stimulus(thing or event specifies functional reaction) that can trigger a state transition.**

- A ***transition* is a relationship between two states**

indicating that an object in the **first state will perform certain actions and enter the second state based on conditions are satisfied.**

- **Graphically, a statechart diagram is a collection of vertices and arcs.**

Common Properties:

A statechart diagram is just a special kind of diagram and

shares the same common properties as do all other diagrams

• What distinguishes a **statechart diagram from all other kinds of diagrams is its content**.

Contents: **Statechart diagrams commonly contain**

Simple states and

composite states.

- **Transitions, including**
- **events and actions**.

Common Uses:

You use statechart diagrams to model the **dynamic aspects of a system.**

- These **dynamic aspects** may **involve** **including classes (which includes active classes), interfaces, components, and nodes.**

- **statechart diagrams in the context of the system as a whole, a subsystem, or a class**.

When you model the **dynamic aspects of a system,** (class, or a use case)

- To model **reactive** objects

A **reactive object is one whose behavior is best characterized by its response** to events dispatched from outside its context.

A **reactive** object is **idle until it receives an event**.

"When it receives an event, its response usually **depends on previous events**.

After the object responds to an event, it becomes idle again, waiting for the next event. "

Common Modeling Technique:

1. Modeling Reactive Objects

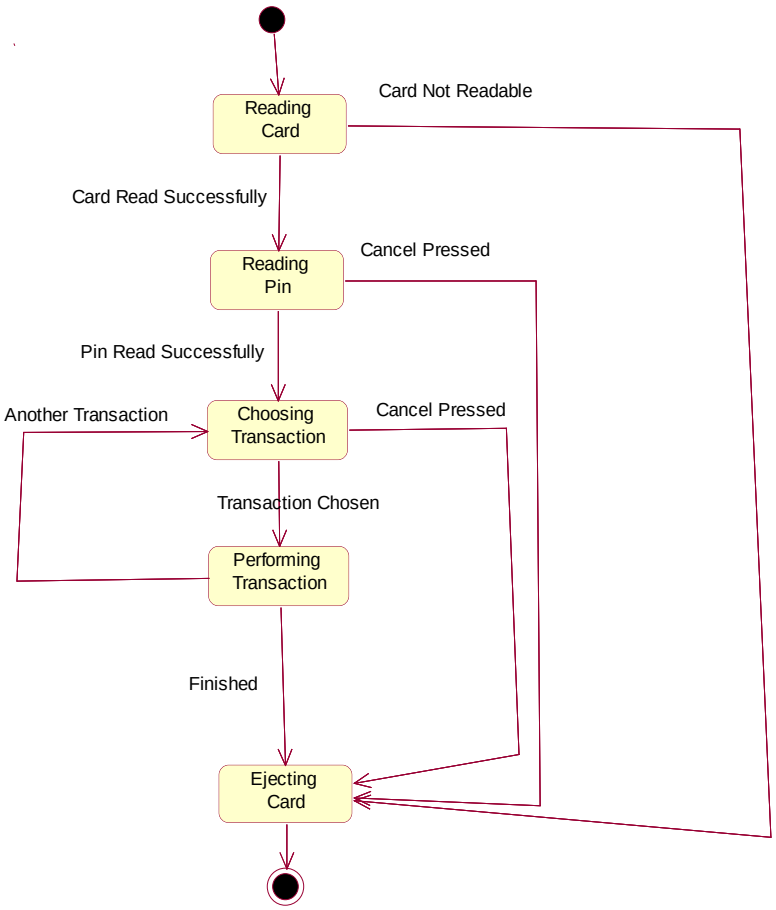
- Choose the **context for the state machine**, **whether it is a system or sub-system.**
- **Choose the initial and final states for the object**.
- possibly state the **pre- and postconditions of the initial and final states,** respectively.
- **Decide on the events that may trigger** a **transition from state to state**.
- Consider ways to **simplify your machine by using substates, branches, forks, joins, and states**.
- **Check that all states** are **reachable under some combination of events**.
- **Trace through the state machine, either manually or by using tools, t**o **expected sequence responces.**

2. Forward and Reverse Engineering

Forward engineering(the creation of code from a model) is possible for statechart diagrams,especially if the context of the diagram is a class. For example, using the previous statechart diagram, a forward engineering tool could generate the following Java code for the class

Reverse engineering (the creation of a model from code) is theoretically possible, but practicallynot very useful. The choice of what constitutes a meaningful state is in the eye of the designer.Reverse engineering tools have no capacity for abstraction and therefore cannot automatically produce meaningful statechart diagrams.

State chart for ATM



State chart for Library information system

